TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM
TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG
KHOA CÔNG NGHỆ THÔNG TIN



TRẦN HỮU NHÂN - 52300235
NGUYỄN NHẬT HÀO - 52300198

XÂY DỰNG NỀN TẢNG GÂY QUỸ CỘNG ĐỒNG CHO CÁC DỰ ÁN XÃ HỘI VÀ KHỞI NGHIỆP

TÀI LIỆU THIẾT KẾ KIẾN TRÚC HỆ THỐNG

ĐỒ ÁN TỐT NGHIỆP

MẠNG MÁY TÍNH VÀ TRUYỀN THÔNG DỮ LIỆU

THÀNH PHỐ HỒ CHÍ MINH, 2026

TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM
TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG
KHOA CÔNG NGHỆ THÔNG TIN



TRẦN HỮU NHÂN - 52300235
NGUYỄN NHẬT HÀO - 52300198

XÂY DỰNG NỀN TẢNG GÂY QUỸ CỘNG ĐỒNG CHO CÁC DỰ ÁN XÃ HỘI VÀ KHỞI NGHIỆP

TÀI LIỆU THIẾT KẾ KIẾN TRÚC HỆ THỐNG

ĐỒ ÁN TỐT NGHIỆP

MẠNG MÁY TÍNH VÀ TRUYỀN THÔNG DỮ LIỆU

Người hướng dẫn

ThS. DƯƠNG HỮU PHƯỚC

THÀNH PHỐ HỒ CHÍ MINH, 2026

XÂY DỰNG NỀN TẢNG GÂY QUỸ CỘNG ĐỒNG CHO CÁC DỰ ÁN XÃ HỘI VÀ KHỞI NGHIỆP
TÓM TẮT TÀI LIỆU THIẾT KẾ KIẾN TRÚC

Tài liệu mô tả kiến trúc của nền tảng gây quỹ cộng đồng Góp Mầm tính đến ngày 28/10/2026. Mục đích là làm cơ sở thống nhất cho việc hiện thực, kiểm thử và triển khai hệ thống, bổ sung cho Đặc tả yêu cầu phần mềm (SRS), Đặc tả ERD và bộ sơ đồ UML của đề tài.

Hệ thống được thiết kế theo mô hình client–server nhiều dịch vụ. Ba ứng dụng độc lập gồm frontend Next.js, backend NestJS và dịch vụ AI FastAPI giao tiếp qua REST API có phiên bản `/api/v1`, đặt sau một reverse proxy (Nginx khi chạy cục bộ, Application Load Balancer trên AWS). Dữ liệu nghiệp vụ lưu trong MySQL 8.4; Redis 7 giữ bộ đếm giới hạn tần suất. Bên trong, backend chia thành 18 module nghiệp vụ, mỗi module theo lớp Controller – Service – Repository – Entity; các tích hợp bên ngoài (thanh toán, email, lưu trữ tệp, đăng nhập Google, AI) được đóng gói qua interface và luôn có bản giả lập để kiểm thử.

Các quyết định chính: backend là nơi duy nhất ghi dữ liệu và cập nhật trạng thái tiền, chỉ sau khi xác minh webhook/IPN; AI chỉ hỗ trợ ra quyết định và có dự phòng hai lớp; cùng một bộ Docker image chạy trên Docker Compose và AWS ECS Fargate (Terraform).

Tài liệu gồm năm chương: giới thiệu; tổng quan kiến trúc; thiết kế chi tiết các thành phần; giao tiếp và bảo mật; triển khai, chất lượng và hạn chế.

Từ khóa: kiến trúc phần mềm, crowdfunding, Next.js, NestJS, FastAPI, MySQL, Docker, AWS.

MỤC LỤC

DANH MỤC HÌNH VẼ **vi**

DANH MỤC BẢNG BIỂU **vii**

DANH MỤC CÁC TỪ VIẾT TẮT **viii**

1 GIỚI THIỆU **1**

1.1 Mục đích tài liệu 1

1.2 Phạm vi 1

1.3 Thuật ngữ 2

1.4 Tài liệu liên quan 2

1.5 Bố cục tài liệu 2

2 TỔNG QUAN KIẾN TRÚC **4**

2.1 Mô hình kiến trúc 4

2.2 Sơ đồ kiến trúc tổng thể 4

2.3 Nguyên tắc thiết kế 4

2.4 Công nghệ sử dụng 5

2.5 Quyết định kiến trúc đáng chú ý 6

3 THIẾT KẾ CHI TIẾT CÁC THÀNH PHẦN **7**

3.1 Frontend 7

3.1.1 Cấu trúc thư mục 7

3.1.2 Điều hướng và kiểm soát truy cập phía giao diện 8

3.2 Backend 9

3.2.1 Kiến trúc phân lớp 9

3.2.2 Cấu trúc thư mục 10

3.2.3 Các module nghiệp vụ 11

3.2.4 Lớp dùng chung 12

3.3 Dịch vụ AI 13
3.4 Cơ sở dữ liệu và bộ nhớ đệm 14
3.5 Tích hợp bên ngoài 14
4 GIAO TIẾP VÀ BẢO MẬT 15
4.1 Các kênh giao tiếp 15
4.2 Tác vụ định kỳ 15
4.3 Luồng xử lý một yêu cầu từ giao diện tới cơ sở dữ liệu 16
4.4 Luồng nghiệp vụ trọng tâm 18
4.4.1 Tài trợ và xác nhận thanh toán 18
4.4.2 Gợi ý dự án và phương án dự phòng AI 18
4.5 Thiết kế bảo mật 19
5 TRIỂN KHAI, CHẤT LƯỢNG VÀ HẠN CHẾ 22
5.1 Triển khai bằng Docker Compose 22
5.2 Triển khai trên AWS 22
5.3 Thuộc tính chất lượng 24
5.4 Hạn chế và hướng mở rộng 24
5.5 Kết luận 25
TÀI LIỆU THAM KHẢO 26

DANH MỤC HÌNH VẼ

2.1 Sơ đồ kiến trúc tổng thể của nền tảng Góp Mầm 5
3.1 Sơ đồ điều hướng giữa các trang chính 9
3.2 Kiến trúc phân lớp bên trong một module backend 10
4.1 Sáu bước xử lý một yêu cầu từ giao diện tới cơ sở dữ liệu 16
4.2 Sơ đồ tuần tự luồng tài trợ và xác nhận thanh toán (SD05) 19
4.3 Sơ đồ tuần tự luồng gợi ý dự án có phương án dự phòng (SD08) 20
5.1 Sơ đồ triển khai trên AWS 23

DANH MỤC BẢNG BIỂU

1.1 Thuật ngữ sử dụng trong tài liệu 2
2.1 Công nghệ sử dụng theo từng tầng 6
3.1 Cấu trúc thư mục frontend 7
3.2 Tương ứng giữa thư mục frontend và module backend 8
3.3 Cấu trúc thư mục backend 10
3.4 Các file trong một module backend (ví dụ module `donations`) 11
3.5 Các module nghiệp vụ của backend 11
3.6 Các endpoint của dịch vụ AI 13
3.7 Cấu trúc thư mục dịch vụ AI 13
3.8 Các gateway tích hợp bên ngoài 14
4.1 Các kênh giao tiếp giữa các thành phần 15
4.2 Các tác vụ định kỳ trong backend 16
4.3 Các bước xử lý khi bấm nút Theo dõi 17
4.4 Các biện pháp bảo mật theo lớp 20
5.1 Các dịch vụ trong Docker Compose 22
5.2 Tài nguyên AWS được tạo bằng Terraform 23
5.3 Thuộc tính chất lượng và cách đáp ứng 24

DANH MỤC CÁC TỪ VIẾT TẮT

Viết tắt	Tiếng Anh
ADR	Architecture Decision Record
AI	Artificial Intelligence
ALB	Application Load Balancer
API	Application Programming Interface
AWS	Amazon Web Services
AZ	Availability Zone
CORS	Cross-Origin Resource Sharing
CSDL	Cơ sở dữ liệu
CSV	Comma-Separated Values
DTO	Data Transfer Object
ECR	Elastic Container Registry
ECS	Elastic Container Service
ERD	Entity Relationship Diagram
HMAC	Hash-based Message Authentication Code
IaC	Infrastructure as Code
IAM	Identity and Access Management
IPN	Instant Payment Notification
JWT	JSON Web Token

ORM	Object-Relational Mapping
OTP	One-Time Password
QR	Quick Response code
RDS	Relational Database Service
REST	Representational State Transfer
S3	Simple Storage Service
SEO	Search Engine Optimization
SMTP	Simple Mail Transfer Protocol
SRS	Software Requirements Specification
SSR	Server-Side Rendering
UUID	Universally Unique Identifier
VPC	Virtual Private Cloud

CHƯƠNG 1: GIỚI THIỆU

1.1 Mục đích tài liệu

Tài liệu mô tả kiến trúc của nền tảng Góp Mầm: các thành phần của hệ thống, trách nhiệm của từng thành phần, cách chúng giao tiếp với nhau, cách hệ thống được bảo mật và triển khai. Đây là cơ sở để hiện thực, kiểm thử và vận hành hệ thống, đồng thời giải thích lý do của các quyết định kỹ thuật chính.

Tài liệu bổ sung cho ba nhóm tài liệu khác của đề tài: Đặc tả yêu cầu phần mềm (SRS) trả lời câu hỏi hệ thống *cần làm gì*; Đặc tả ERD mô tả *dữ liệu* được lưu như thế nào; các sơ đồ UML mô tả *hành vi* của từng chức năng. Tài liệu kiến trúc trả lời câu hỏi hệ thống *được tổ chức* như thế nào để đáp ứng các yêu cầu đó.

1.2 Phạm vi

Phạm vi tài liệu bao gồm toàn bộ mã nguồn trong kho `DATN_2627_Website`:

- `frontend/`: ứng dụng web Next.js.
- `backend/`: API nghiệp vụ NestJS, migration và dữ liệu mẫu.
- `ai-service/`: dịch vụ suy luận AI FastAPI và pipeline huấn luyện ngoại tuyến.
- `infra/`: cấu hình Docker Compose, Nginx và Terraform cho AWS.
- `docs/`: tài liệu kỹ thuật đi kèm mã nguồn (hợp đồng API, ADR, cơ sở dữ liệu, vận hành, kiểm thử).
- `Document/`: bộ tài liệu đồ án (phân tích, thiết kế, sơ đồ UML, hướng dẫn sử dụng).
- `scripts/`: lệnh chạy nhanh Node.js, npm và Python trên Windows.

Thiết kế giao diện chi tiết, đặc tả từng bảng dữ liệu và kịch bản của từng use case nằm ở các tài liệu riêng và chỉ được nhắc tới khi cần.

1.3 Thuật ngữ

Các thuật ngữ chuyên môn dùng trong tài liệu được giải thích trong Bảng 1.1. Các từ viết tắt được liệt kê ở Danh mục các từ viết tắt.

Bảng 1.1 Thuật ngữ sử dụng trong tài liệu

Thuật ngữ	**Ý nghĩa**
Server-Side Rendering	Trang được render ở máy chủ trước khi gửi về trình duyệt, giúp công cụ tìm kiếm đọc được nội dung
REST API	Giao diện lập trình qua HTTP, có phiên bản `/api/v1`
Webhook / IPN	Lời gọi từ cổng thanh toán về backend để báo kết quả giao dịch
Idempotency key	Khóa do client gửi kèm để một yêu cầu tạo giao dịch gửi lại nhiều lần chỉ được xử lý một lần
Fallback	Phương án dự phòng khi mô hình AI hoặc dịch vụ AI không sẵn sàng
Reverse proxy	Máy chủ trung gian nhận mọi yêu cầu từ người dùng rồi chuyển tới dịch vụ phù hợp
Cron	Tác vụ chạy định kỳ theo lịch trong backend
Human-in-the-loop	Kết quả AI chỉ là gợi ý, quyết định cuối cùng do con người đưa ra

1.4 Tài liệu liên quan

- Đặc tả yêu cầu phần mềm: `Document/01_TaiLieuPhanTich/01_SRS/SRS_GopMam.docx`.
- Danh sách Actor và Đặc tả Use Case: `Document/01_TaiLieuPhanTich/`.
- Đặc tả ERD và mô hình quan hệ: `Document/02_TaiLieuThietKe/01_ERD/`, `docs/database/schema.dbml`.
- Sơ đồ UML (Use Case, Activity, Sequence, Class): `Document/03_SoDoUML/`.
- Hợp đồng API OpenAPI 3: `docs/api/openapi.yaml`.
- ADR-001 – chọn NestJS cho backend: `docs/architecture/adr-001-backend-stack.md`.

1.5 Bố cục tài liệu

- **Chương 1 – Giới thiệu:** mục đích, phạm vi, thuật ngữ và tài liệu liên quan.
- **Chương 2 – Tổng quan kiến trúc:** mô hình kiến trúc, sơ đồ tổng thể, nguyên

tắc thiết kế và công nghệ sử dụng.

- **Chương 3 – Thiết kế chi tiết các thành phần:** cấu trúc thư mục và vai trò của frontend, backend, dịch vụ AI; điều hướng phía giao diện; cơ sở dữ liệu và các tích hợp bên ngoài.
- **Chương 4 – Giao tiếp và bảo mật:** các kênh giao tiếp, tác vụ định kỳ, luồng xử lý một yêu cầu từ giao diện tới cơ sở dữ liệu, luồng nghiệp vụ trọng tâm và thiết kế bảo mật.
- **Chương 5 – Triển khai, chất lượng và hạn chế:** kiến trúc triển khai trên Docker Compose và AWS, thuộc tính chất lượng, hạn chế và hướng mở rộng.

CHƯƠNG 2: TỔNG QUAN KIẾN TRÚC

2.1 Mô hình kiến trúc

Hệ thống theo mô hình client–server nhiều dịch vụ. Ba ứng dụng độc lập là frontend, backend và dịch vụ AI; mỗi ứng dụng có mã nguồn, Dockerfile và vòng đời riêng, giao tiếp với nhau qua REST API có phiên bản `/api/v1`. Toàn bộ yêu cầu từ người dùng đi qua một reverse proxy: Nginx khi chạy bằng Docker Compose, Application Load Balancer khi chạy trên AWS.

Bên trong backend áp dụng kiến trúc phân lớp theo module nghiệp vụ. Cách chia này giữ được ranh giới nghiệp vụ rõ ràng như một hệ thống microservice nhưng vẫn triển khai đơn giản như một ứng dụng duy nhất, phù hợp với quy mô đồ án hai thành viên.

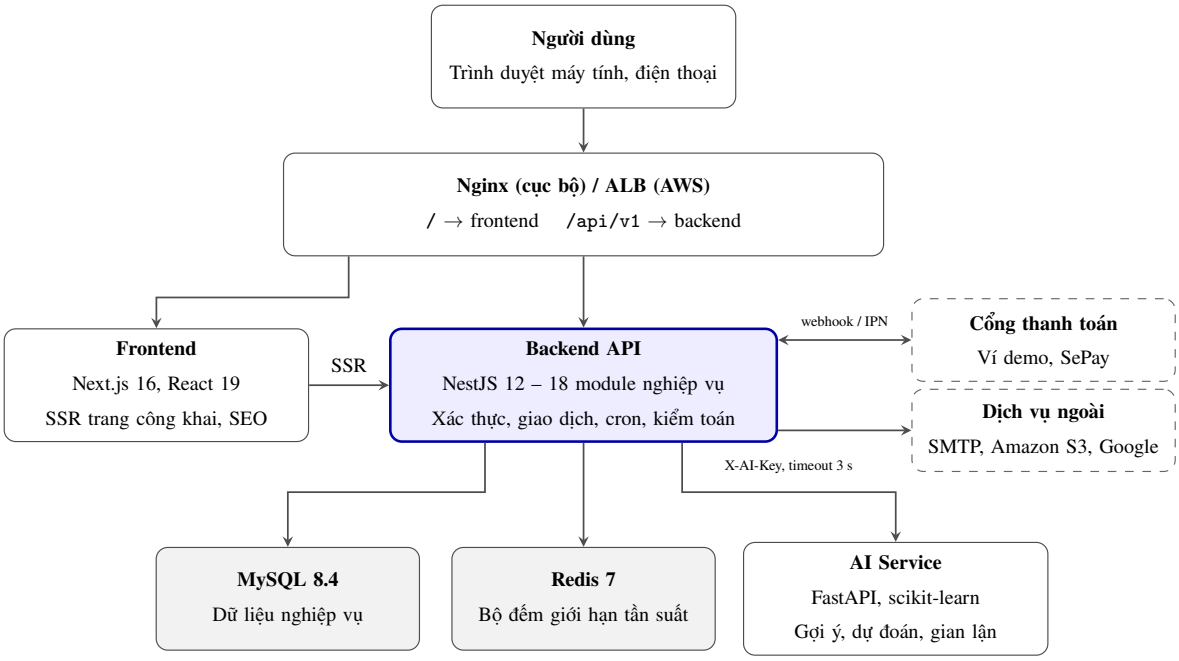
2.2 Sơ đồ kiến trúc tổng thể

Hình 2.1 trình bày các thành phần của hệ thống và chiều gọi giữa chúng.

Người dùng chỉ đi qua reverse proxy. Backend là thành phần duy nhất ghi vào MySQL, gọi dịch vụ AI và nhận webhook/IPN từ cổng thanh toán. Frontend khi render phía máy chủ gọi thẳng backend trong mạng nội bộ; trình duyệt gọi backend qua reverse proxy. Trên AWS, dịch vụ AI không được mở ra load balancer, chỉ backend gọi được.

2.3 Nguyên tắc thiết kế

- **Ranh giới rõ ràng:** nghiệp vụ, phân quyền và trạng thái tiền chỉ nằm ở backend. Frontend chỉ ẩn/hiện giao diện và không được tin cậy.
- **Số liệu truy vết được:** số tiền đã huy động và số liệu quỹ được tính từ giao dịch đã xác minh; mọi thay đổi quan trọng được ghi nhật ký kiểm toán.
- **AI chỉ hỗ trợ ra quyết định:** kết quả AI không tự khóa, từ chối hay xóa dữ



Hình 2.1 Sơ đồ kiến trúc tổng thể của nền tảng Góp Mầm

liệu; luôn có phương án dự phòng khi AI không sẵn sàng.

- **Tích hợp qua interface:** thanh toán, email, lưu trữ tệp, đăng nhập Google và AI đều có bản giả lập để phát triển và kiểm thử mà không cần dịch vụ thật.
- **Không lưu trạng thái phiên:** xác thực bằng JWT nên có thể chạy nhiều instance backend phía sau load balancer.
- **Giống nhau ở mọi môi trường:** cùng một bộ Docker image chạy trên máy cá nhân và trên AWS; cấu hình khác nhau chỉ nằm ở biến môi trường.

2.4 Công nghệ sử dụng

Phần web dùng chung TypeScript cho cả frontend và backend; phần AI tách riêng bằng Python vì hệ sinh thái học máy của Python đầy đủ nhất. Bảng 2.1 tổng hợp công nghệ theo từng tầng, phiên bản lấy từ `package.json` và `requirements.txt` tại thời điểm viết tài liệu.

Bảng 2.1 Công nghệ sử dụng theo từng tầng

Tầng	**Công nghệ, phiên bản**	**Lý do lựa chọn**
Frontend	Next.js 16.3, React 19.2, TypeScript	SSR và App Router tốt cho SEO trang dự án; cùng ngôn ngữ với backend
Backend	NestJS 12, TypeORM 1.1, class-validator	Module và dependency injection; guard, pipe, interceptor có sẵn cho phân quyền, kiểm tra đầu vào và kiểm toán
Xác thực	@nestjs/jwt, passport-jwt, bcrypt, google-auth-library	JWT không trạng thái; xác minh ID token Google phía máy chủ
Tác vụ định kỳ	@nestjs/schedule 12	Chốt chiến dịch hết hạn, hết hạn đơn chờ, nhắc mốc tiến độ
Dịch vụ AI	Python 3.12, FastAPI 0.141, scikit-learn 1.9.1, pandas, NumPy	API suy luận nhẹ; scikit-learn ghim đúng phiên bản dùng khi huấn luyện mô hình
Cơ sở dữ liệu	MySQL 8.4	Quan hệ, giao dịch ACID cho dữ liệu tiền; tương thích MariaDB trên hosting
Bộ nhớ đệm	Redis 7 (ioredis)	Bộ đếm giới hạn tần suất dùng chung giữa các instance
Thanh toán	Ví demo (sandbox), SePay	Kiểm thử không phát sinh tiền thật; SePay hỗ trợ QR chuyển khoản nội địa
Email, lưu trữ	Nodemailer (SMTP), ổ đĩa cục bộ hoặc Amazon S3	Đổi nhà cung cấp bằng biến môi trường
Hạ tầng	Docker Compose, Nginx 1.27, Terraform (AWS)	Môi trường chạy giống nhau từ máy cá nhân tới đám mây; hạ tầng mô tả bằng mã
Tài liệu API	OpenAPI 3 / Swagger	Hợp đồng giữa frontend và backend

2.5 Quyết định kiến trúc đáng chú ý

Đề cương ban đầu (ADR-001) dự kiến dùng Java Spring Boot cho backend. Trong quá trình thực hiện, nhóm chuyển sang NestJS để frontend và backend dùng chung một ngôn ngữ, giảm số công cụ phải duy trì. Ranh giới nghiệp vụ, phân quyền và giao dịch vẫn được giữ ở backend theo kiến trúc nhiều lớp như thiết kế ban đầu. Dịch vụ AI vẫn viết bằng Python và tách thành dịch vụ riêng.

CHƯƠNG 3: THIẾT KẾ CHI TIẾT CÁC THÀNH PHẦN

Mỗi thành phần có vòng đời và Dockerfile riêng. Chỉ backend được ghi vào cơ sở dữ liệu và gọi dịch vụ AI; các thành phần còn lại giao tiếp với dữ liệu thông qua backend.

3.1 Frontend

Frontend dùng Next.js App Router. Trang công khai (trang chủ, danh sách và chi tiết dự án, trang tổ chức) được render phía máy chủ để hỗ trợ SEO, Open Graph, sitemap và JSON-LD; các trang tương tác (tạo chiến dịch, ủng hộ, quản trị) được render phía trình duyệt.

3.1.1 Cấu trúc thư mục

Mã nguồn frontend nằm trong `frontend/src/` và chia thành bốn thư mục chính theo vai trò (Bảng 3.1).

Bảng 3.1 Cấu trúc thư mục frontend

Thư mục	**Vai trò**	**Ví dụ**
`src/app/`	Định tuyến: mỗi thư mục con chứa `page.tsx` là một địa chỉ trang (URL tiếng Việt). Thư mục trong ngoặc vuông là phần thay đổi của URL. File ở đây chủ yếu lấy dữ liệu và ghép các thành phần từ `features/`	`du-an/[slug]/page.tsx` → `/du-an/<slug>`; `admin/[section]/` → `/admin/cho-duyet`
`src/features/`	Chức năng theo nghiệp vụ, 21 nhóm. Mỗi nhóm chứa form, nút, danh sách cùng state và xử lý sự kiện của nghiệp vụ đó	`donations/DonationPanel.tsx`, `follows/FollowCampaignButton.tsx`
`src/components/`	Thành phần giao diện dùng chung, không gắn với một nghiệp vụ	`layout/SiteHeader.tsx`, `ui/Icon.tsx`
`src/lib/`	Mã không có giao diện: hàm gọi API, trạng thái đăng nhập, định dạng tiền VND và ngày, chính sách mật khẩu	`api.ts`, `api/donations.ts`, `auth.ts`, `format.ts`

Thư mục `src/lib/api/` có một file cho mỗi nhóm API của backend (`campaigns.ts`,

donations.ts, follows.ts...). Mọi file này đều gọi qua một hàm chung apiFetch() trong src/lib/api.ts. Hàm này gắn access token vào header, tự xin token mới khi nhận mã 401, và chuyển lỗi máy chủ thành ApiError để giao diện hiển thị.

Tên nhóm trong features/ khớp với tên module của backend, nên từ một chức năng trên giao diện có thể tìm ngay phần xử lý tương ứng ở máy chủ (Bảng 3.2).

Bảng 3.2 Tương ứng giữa thư mục frontend và module backend

Nghiệp vụ	**Frontend**	**Backend**
Xác thực	features/auth/, lib/auth.ts	modules/auth/
Chiến dịch	features/campaigns/, features/campaign-create/, lib/api/campaigns.ts	modules/campaigns/
Ủng hộ	features/donations/, lib/api/donations.ts	modules/donations/
Theo dõi	features/follows/, lib/api/follows.ts	modules/follows/
Bình luận	features/comments/, lib/api/comments.ts	modules/comments/
Tổ chức	features/organizations/, lib/api/organizations.ts	modules/organizations/
Tài chính	features/finance/, lib/api/finance.ts	modules/finance/
Quản trị	features/admin/, lib/api/admin.ts	modules/admin/

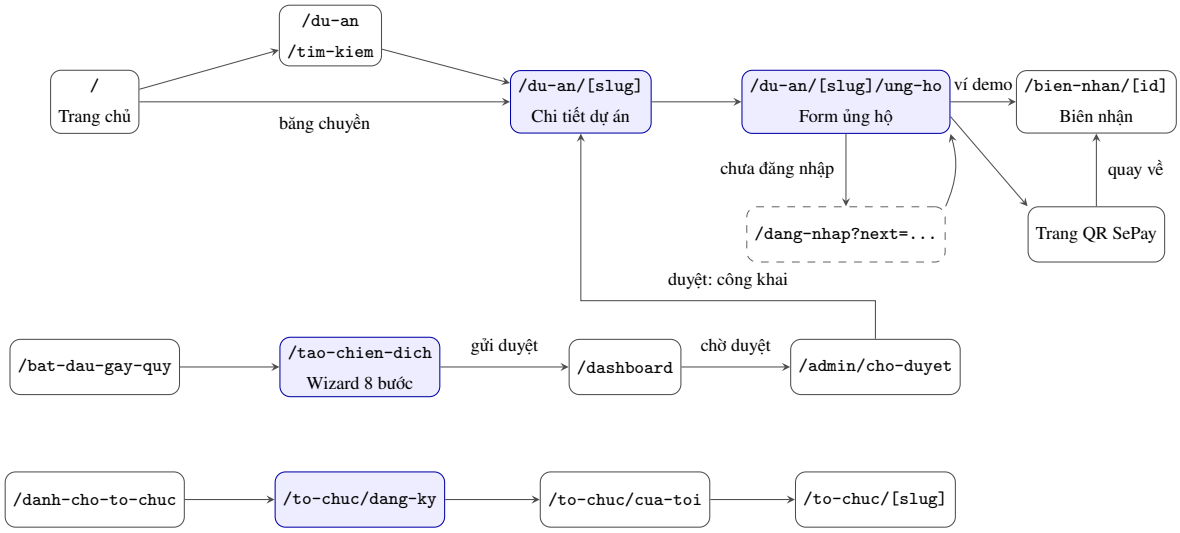
Khi render phía máy chủ, frontend gọi thẳng backend trong mạng nội bộ qua biến API_INTERNAL_URL; trình duyệt gọi qua đường dẫn tương đối /api/v1 đi qua reverse proxy.

3.1.2 Điều hướng và kiểm soát truy cập phía giao diện

Các trang được nối với nhau theo năm luồng chính (Hình 3.1). Đầu header và cuối footer là các điểm vào chung của mọi luồng.

Giao diện áp dụng hai quy tắc điều hướng chung cho mọi luồng:

- **Chuyển sang đăng nhập rồi quay lại:** thao tác cần tài khoản (ủng hộ, theo dõi, bình luận, báo cáo vi phạm, tạo chiến dịch) chuyển người dùng sang /dang-nhap?next=<trang hiện tại>. Đăng nhập xong, người dùng được



Hình 3.1 Sơ đồ điều hướng giữa các trang chính

đưa về đúng trang cũ. Hàm `safeNext()` chỉ chấp nhận đường dẫn nội bộ bắt đầu bằng `/` và từ chối `//host` hay `https://...`, nhằm chống lỗ hổng chuyển hướng mở (open redirect).

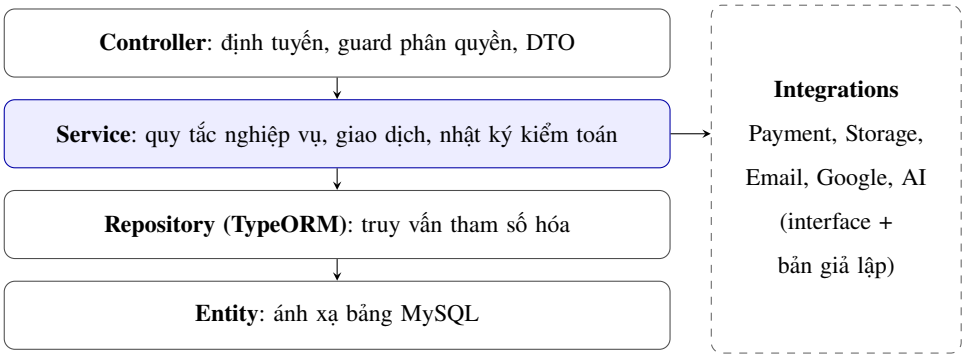
- **Tài khoản quản trị chỉ làm việc trong `/admin`:** quản trị viên đăng nhập luôn được đưa vào `/admin` và bỏ qua tham số `next`. Khi mở trang dành cho người dùng thường (`/tao-chien-dich`, `/dashboard`, `/to-chuc/dang-ky`, `/to-chuc/cua-toi`), hook `useRedirectAdmin()` chuyển họ về `/admin`. Header của quản trị viên cũng không hiện mục tạo chiến dịch.

Các quy tắc trên chỉ giúp giao diện gọn và đúng ngữ cảnh. Quyền thật sự được kiểm tra lại ở backend bằng guard (Mục 4): ví dụ `NonAdminGuard` từ chối yêu cầu ủng hộ hay theo dõi của quản trị viên dù yêu cầu đó được gửi từ đâu.

3.2 Backend

3.2.1 Kiến trúc phân lớp

Backend NestJS chia theo module nghiệp vụ. Mỗi module tổ chức theo bốn lớp như Hình 3.2: controller nhận yêu cầu và DTO đã được kiểm tra, service giữ quy tắc nghiệp vụ, repository (TypeORM) truy vấn tham số hóa, entity ánh xạ bảng dữ liệu.



Hình 3.2 Kiến trúc phân lớp bên trong một module backend

3.2.2 Cấu trúc thư mục

Mã nguồn backend nằm trong `backend/src/` (Bảng 3.3). Ngoài `src/`, thư mục `backend/database/migrations/` chứa các bước tạo và sửa bảng theo thứ tự, còn `backend/test/` chứa kiểm thử tự động.

Bảng 3.3 Cấu trúc thư mục backend

Thư mục, file	**Vai trò**
`main.ts`	Điểm khởi động: tạo ứng dụng, đặt tiền tố `/api/v1`, bật `ValidationPipe`, helmet, CORS
`app.module.ts`	Module gốc, khai báo mọi module nghiệp vụ và kết nối cơ sở dữ liệu
`modules/`	18 module nghiệp vụ (Bảng 3.5)
`common/`	Thành phần dùng chung: nhật ký kiểm toán, giới hạn tần suất, decorator, filter, interceptor, validator
`integrations/`	Gateway tới dịch vụ bên ngoài: thanh toán, email, lưu trữ, Google, AI (Mục 3.5)
`config/`	Đọc biến môi trường thành cấu hình ứng dụng, cơ sở dữ liệu và AI

Mọi module trong `modules/` theo cùng một khuôn file, ứng với các lớp ở Hình 3.2. Bảng 3.4 lấy module `donations` làm ví dụ.

Guard dùng chung cho mọi module nằm trong `modules/auth/guards/`: `JwtAuthGuard` yêu cầu đăng nhập, `OptionalJwtAuthGuard` nhận cả khách lẫn người đã đăng nhập, `NonAdminGuard` từ chối tài khoản quản trị ở các thao tác dành cho người dùng thường, `RolesGuard` kiểm tra vai trò, `StepUpGuard` yêu cầu mã OTP trước thao tác nhạy cảm.

Bảng 3.4 Các file trong một module backend (ví dụ module `donations`)

File	Lớp	Vai trò
`donations.controller.ts`	Controller	Khai báo địa chỉ API (`@Post`, `@Get`), gắn guard phân quyền, nhận DTO đã kiểm tra rồi gọi service
`donations.service.ts`	Service	Quy tắc nghiệp vụ: kiểm tra điều kiện, giao dịch cơ sở dữ liệu, ghi nhật ký kiểm toán, gửi thông báo
`dto/donation.dto.ts`	DTO	Mô tả dữ liệu đầu vào hợp lệ (kiểu, độ dài, giá trị cho phép) bằng class-validator
`entities/donation.entity.ts`	Entity	Ánh xạ bảng `donations` trong MySQL: cột, kiểu, quan hệ
`donations.module.ts`	–	Gom controller, service, entity và khai báo phụ thuộc với NestJS
`sepay-ipn.controller.ts`, `donation-expiry.service.ts`	Controller, Service	File riêng cho phần đặc thù của module: nhận IPN SePay, tác vụ định kỳ hết hạn đơn chờ

3.2.3 Các module nghiệp vụ

Backend gồm 18 module, trách nhiệm của từng module được nêu trong Bảng 3.5.

Bảng 3.5 Các module nghiệp vụ của backend

Module	**Trách nhiệm chính**
`auth`	Đăng ký, đăng nhập JWT, refresh token xoay vòng, đăng nhập Google, xác minh email, đặt lại mật khẩu, OTP cho thao tác nhạy cảm
`users`	Hồ sơ, vai trò, khóa và mở tài khoản
`campaigns`	Vòng đời chiến dịch, ma trận chuyển trạng thái, cron chốt chiến dịch, thống kê lượt xem
`categories`	Lĩnh vực chiến dịch do quản trị viên quản lý
`donations`	Tạo giao dịch với idempotency key, nhận IPN SePay, sổ cái công khai, cron hết hạn đơn chờ quá 30 phút
`payments`	Webhook ví demo có chữ ký HMAC, chống phát lại

Module	Trách nhiệm chính
`finance`	Hoàn tiền, đối soát, biên nhận điện tử, xuất CSV
`rewards`	Mức quà tặng và số suất còn lại
`progress`	Mốc tiến độ, bài cập nhật, chứng từ chi tiêu, lịch sử thay đổi kế hoạch, cron nhắc mốc quá hạn
`organizations`	Hồ sơ và xác minh tổ chức thụ hưởng
`comments`	Bình luận và hỏi đáp theo luồng
`follows`	Theo dõi dự án
`moderation`	Báo cáo vi phạm và hàng đợi xử lý
`notifications`	Thông báo trong ứng dụng và email
`media`	Tải ảnh lên, kiểm tra loại và dung lượng
`ai`	Gợi ý dự án, dự đoán khả năng thành công, ghi hành vi khi người dùng đồng ý
`admin`	Dashboard, cảnh báo rủi ro, dự án nổi bật, sao lưu và khôi phục dữ liệu
`health`	Kiểm tra trạng thái sống cho Docker và load balancer

3.2.4 Lớp dùng chung

Thư mục `src/common/` chứa các thành phần dùng chung cho mọi module:

- `audit`: ghi nhật ký kiểm toán gồm thời điểm, người thực hiện, giá trị trước và sau.
- `rate-limit`: giới hạn tần suất dựa trên Redis, tự lùi về bộ nhớ trong khi Redis lỗi.
- `html`: làm sạch HTML mô tả chiến dịch trước khi lưu.
- `validators`: kiểm tra mật khẩu mạnh và URL ảnh hợp lệ.

3.3 Dịch vụ AI

Dịch vụ AI chỉ làm nhiệm vụ suy luận. Việc huấn luyện chạy ngoại tuyến (thư mục `training/`, trên Google Colab hoặc Kaggle), kết quả là các file mô hình `.pkl` được nạp từ thư mục `models/`. Dịch vụ không truy cập cơ sở dữ liệu: backend gửi kèm dữ liệu cần thiết (danh sách chiến dịch ứng viên, lịch sử hành vi đã được đồng ý) trong mỗi yêu cầu. Các endpoint được liệt kê trong Bảng 3.6.

Bảng 3.6 Các endpoint của dịch vụ AI

Endpoint	**Chức năng**	**Dự phòng khi không nạp được mô hình**
`/api/v1/recommend`	Gợi ý dự án theo sở thích và hành vi người dùng	Xếp hạng theo mức phổ biến
`/api/v1/predict`	Xác suất chiến dịch đạt mục tiêu, kèm giải thích các yếu tố ảnh hưởng	Luật kinh nghiệm, đánh dấu `fallback=true`
`/api/v1/fraud`	Điểm rủi ro gian lận kèm bằng chứng	Chỉ dùng luật nghiệp vụ
`/api/v1/health`	Trạng thái dịch vụ và mô hình	–

Cấu trúc thư mục của dịch vụ AI được trình bày trong Bảng 3.7.

Bảng 3.7 Cấu trúc thư mục dịch vụ AI

Thư mục	**Vai trò**
`app/main.py`	Điểm khởi động ứng dụng FastAPI
`app/api/v1/`	Các endpoint ở Bảng 3.6
`app/core/`	Cấu hình và bảo mật, gồm kiểm tra header `X-AI-Key` (`security.py`)
`app/services/`	Logic suy luận: `recommender.py` (gợi ý), `predictor.py` (dự đoán thành công), `fraud_detector.py` (gian lận), `model_status.py` (trạng thái nạp mô hình)
`app/schemas/`	Mô tả dữ liệu vào và ra của từng endpoint
`models/`	File mô hình đã huấn luyện (`.pkl`) và chỉ số đánh giá (`.json`)
`training/`, `notebooks/`	Mã huấn luyện ngoại tuyến và notebook phân tích
`tests/`	Kiểm thử tự động

File mô hình `.pkl` không được đưa vào Git, vì vậy một môi trường mới sẽ chạy bằng luật dự phòng cho tới khi mô hình được huấn luyện lại và đặt vào thư mục `models/`.

3.4 Cơ sở dữ liệu và bộ nhớ đệm

MySQL 8.4 lưu toàn bộ dữ liệu nghiệp vụ; chi tiết từng bảng được trình bày trong Đặc tả ERD. Thiết kế dữ liệu tuân theo các nguyên tắc:

- Khóa chính dạng UUID.
- Xóa mềm cho các dữ liệu quan trọng để có thể truy vết.
- Schema chỉ thay đổi qua migration có thứ tự và có rollback, không dùng chế độ tự đồng bộ của ORM.
- Tương thích cả MySQL 8 và MariaDB để có thể đưa lên hosting thông thường.

Redis 7 chỉ giữ bộ đếm giới hạn tần suất. Mất dữ liệu Redis không làm mất dữ liệu nghiệp vụ.

3.5 Tích hợp bên ngoài

Mỗi tích hợp nằm trong `backend/src/integrations/` dưới dạng một interface kèm ít nhất một bản giả lập, nên có thể đổi nhà cung cấp mà không sửa code nghiệp vụ (Bảng 3.8).

Bảng 3.8 Các gateway tích hợp bên ngoài

Gateway	**Bản triển khai**	**Cấu hình chuyển đổi**
`PaymentGateway`	`DemoWalletGateway` (sandbox), SePay	`SEPAY_ENV`, `SEPAY_MERCHANT_ID`; môi trường production tự tắt ví demo
`StorageGateway`	`LocalStorageGateway`, `S3StorageGateway`	`STORAGE_DRIVER=local` hoặc `s3`
`EmailGateway`	SMTP qua Nodemailer hoặc chỉ ghi log	Để trống `SMTP_HOST` thì chỉ ghi log
`GoogleIdentityGateway`	Xác minh ID token Google	Để trống `GOOGLE_CLIENT_ID` thì ẩn nút Google
`AiGateway`	REST tới dịch vụ AI, timeout 3 giây	`AI_SERVICE_URL`, `AI_TIMEOUT_MS`, `AI_API_KEY`

CHƯƠNG 4: GIAO TIẾP VÀ BẢO MẬT

4.1 Các kênh giao tiếp

Mọi giao tiếp giữa các thành phần là HTTP/REST đồng bộ, trừ hai trường hợp: webhook/IPN do cổng thanh toán chủ động gọi về backend và các tác vụ định kỳ chạy bên trong backend. Bảng 4.1 liệt kê từng kênh cùng cách xác thực.

Bảng 4.1 Các kênh giao tiếp giữa các thành phần

Từ	**Đến**	**Giao thức**	**Xác thực**
Trình duyệt	Reverse proxy	HTTP(S)	–
Reverse proxy	Frontend (`/`)	HTTP proxy	–
Reverse proxy	Backend (`/api/`)	HTTP proxy, thêm `X-Request-ID`, `X-Forwarded-*`	JWT Bearer
Frontend (SSR)	Backend	REST trong mạng nội bộ	JWT của người dùng (nếu có)
Backend	Dịch vụ AI	REST nội bộ, timeout 3 giây	Header `X-AI-Key`
Backend	MySQL	Giao thức MySQL qua TypeORM	Tài khoản ứng dụng riêng
Backend	Redis	RESP qua ioredis	Mạng nội bộ
Ví demo	Backend	Webhook	Chữ ký HMAC, timestamp lệch tối đa 5 phút
SePay	Backend (`/api/v1/payments/sepay/ipn`)	IPN	`X-Secret-Key`, đối chiếu số tiền
Backend	SePay	REST tra cứu đơn	Khóa merchant
Backend	SMTP, S3, Google	SMTP, HTTPS	Thông tin xác thực trong biến môi trường

4.2 Tác vụ định kỳ

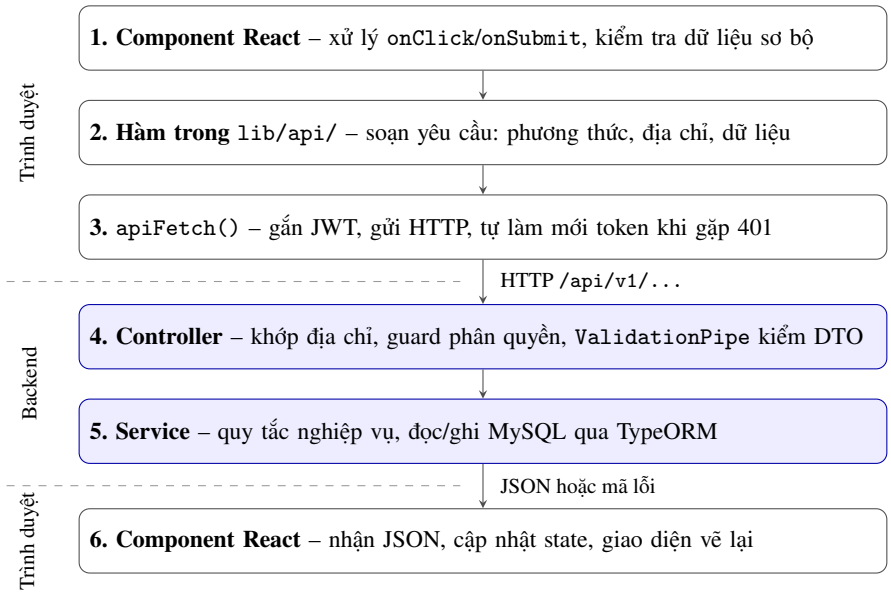
Ba tác vụ định kỳ chạy trong process backend bằng `@nestjs/schedule` (Bảng 4.2). Các tác vụ đều cập nhật dữ liệu có điều kiện, chỉ đổi trạng thái khi bản ghi vẫn ở trạng thái mong đợi tại thời điểm ghi, để không xung đột với thao tác đồng thời của người dùng hay quản trị viên.

Bảng 4.2 Các tác vụ định kỳ trong backend

Tác vụ	Chu kỳ	Công việc
`CampaignExpiryService`	Mỗi giờ	Chiến dịch đang gây quỹ đã hết hạn chuyển sang Thành công hoặc Không đạt mục tiêu
`DonationExpiryService`	Mỗi 10 phút	Giao dịch chờ thanh toán quá 30 phút chuyển sang Hết hạn
`MilestoneOverdueService`	Mỗi giờ	Đánh dấu mốc chậm tiến độ và nhắc chủ dự án

4.3 Luồng xử lý một yêu cầu từ giao diện tới cơ sở dữ liệu

Mọi thao tác của người dùng có ghi dữ liệu (theo dõi dự án, bình luận, ủng hộ, gửi duyệt chiến dịch...) đều đi qua sáu bước như Hình 4.1. Bước 1–3 và bước 6 chạy trong trình duyệt; bước 4–5 chạy ở backend. Hai phía chỉ trao đổi với nhau qua HTTP. Riêng thao tác bấm liên kết chỉ đổi trang mà không qua bước 2–5; trang mới tự gọi API để lấy dữ liệu theo cùng các bước này.



Hình 4.1 Sáu bước xử lý một yêu cầu từ giao diện tới cơ sở dữ liệu

Bảng 4.3 lần theo sáu bước khi người dùng bấm nút *Theo dõi* trên trang chi tiết dự án.

Bảng 4.3 Các bước xử lý khi bấm nút Theo dõi

Bước	**File**	**Xử lý**
1	`frontend/src/features/follows/FollowCampaignButton.tsx`	Hàm toggle khóa nút, chọn theo dõi hoặc bỏ theo dõi tùy trạng thái hiện tại
2	`frontend/src/lib/api/follows.ts`	`followCampaign(id)` tạo yêu cầu `POST /campaigns/:id/follow`
3	`frontend/src/lib/api.ts`	`apiFetch()` gắn `Authorization: Bearer <token>` và gửi tới `/api/v1/campaigns/:id/follow`
4	`backend/src/modules/follows/follows.controller.ts`	`@Post("campaigns/:id/follow")` với `JwtAuthGuard` (phải đăng nhập), `NonAdminGuard` (không phải quản trị viên), `RateLimitGuard` (giới hạn tần suất); `ParseUUIDPipe` kiểm tra mã dự án
5	`backend/src/modules/follows/follows.service.ts`	`follow()` từ chối nếu người dùng là chủ dự án hoặc dự án đã kết thúc; chèn bản ghi vào bảng `campaign_follows` bằng `INSERT IGNORE` để bấm hai lần không lỗi trùng khóa; trả về trạng thái theo dõi và số người theo dõi
6	`FollowCampaignButton.tsx`	`setStatus(next)` cập nhật state; React vẽ lại nút thành *Đang theo dõi* và số người theo dõi mới. Nếu có lỗi, `ApiError` được bắt và hiển thị dưới nút

Hàm `apiFetch()` ở bước 3 dùng chung cho mọi yêu cầu nên các hành vi sau áp dụng thống nhất trên toàn hệ thống:

- **Access token hết hạn:** khi nhận mã 401, hàm dùng refresh token xin cặp token mới đúng một lần rồi gửi lại yêu cầu. Nhiều yêu cầu cùng gặp 401 chỉ dùng chung một lần làm mới. Làm mới thất bại thì xóa phiên đăng nhập.
- **Thao tác nhạy cảm:** khi backend yêu cầu xác minh bổ sung, hàm xin mã OTP từ người dùng rồi gửi lại yêu cầu kèm mã đó.
- **Lỗi nghiệp vụ:** các mã lỗi khác được chuyển thành `ApiError` có mã HTTP và thông báo, để component hiển thị cho người dùng.

Luồng ủng hộ ở Mục 4.4.1 cũng đi qua đúng sáu bước này. Điểm khác là bước 5 tách thành hai pha: tạo đơn ở trạng thái chờ thanh toán, sau đó chỉ cộng tiền khi cổng thanh toán xác nhận.

4.4 Luồng nghiệp vụ trọng tâm

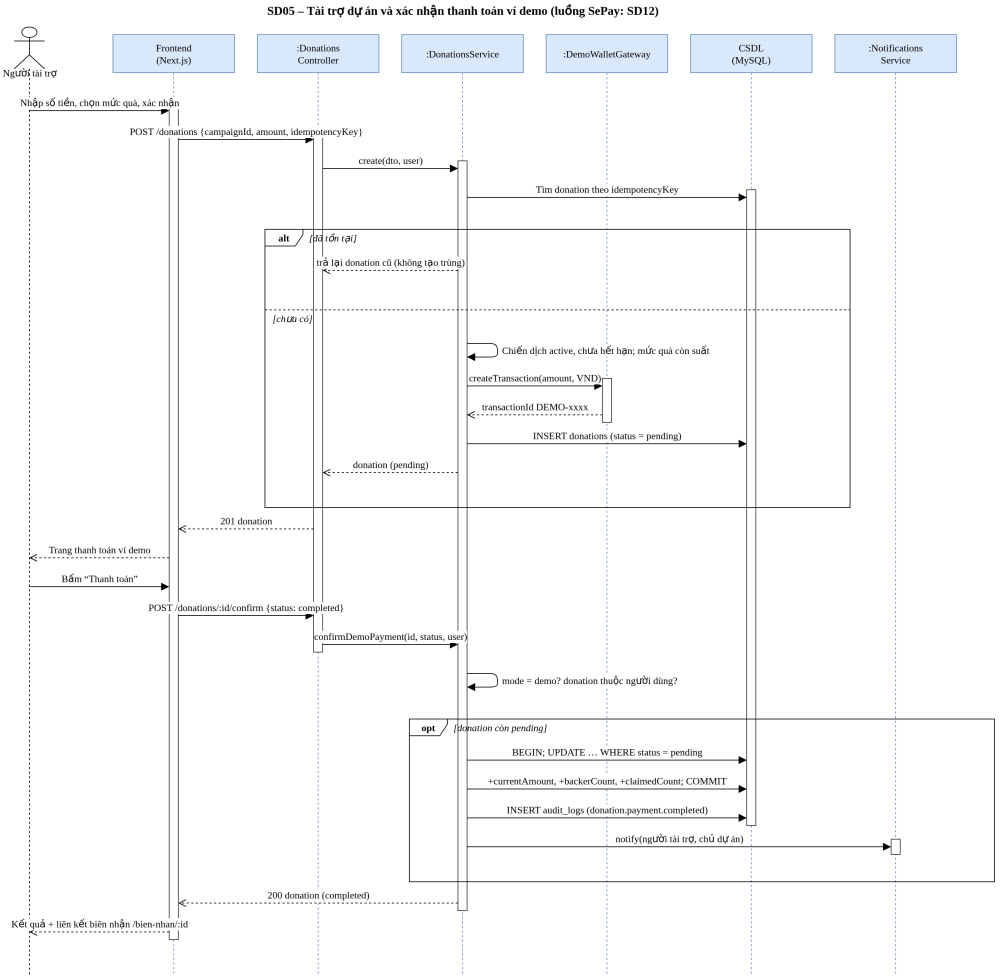
4.4.1 Tài trợ và xác nhận thanh toán

Đây là luồng quan trọng nhất về mặt kiến trúc vì liên quan trực tiếp tới tiền. Trình duyệt không bao giờ tự báo thanh toán thành công. Backend chỉ đổi trạng thái giao dịch sau khi xác minh lời gọi từ cổng thanh toán: chữ ký HMAC với ví demo, khóa `X-Secret-Key` và đối chiếu số tiền với IPN của SePay. Nếu IPN bị lỡ, backend tra cứu lại đơn qua API của SePay. Idempotency key ngăn tạo trùng giao dịch khi người dùng bấm gửi nhiều lần. Trình tự chi tiết được thể hiện trong Hình 4.2.

4.4.2 Gợi ý dự án và phương án dự phòng AI

Kết quả AI có phương án dự phòng ở hai lớp:

1. **Trong dịch vụ AI:** không nạp được file mô hình thì dùng luật kinh nghiệm; phản hồi mang cờ `fallback=true`.
2. **Trong backend:** dịch vụ AI lỗi hoặc phản hồi quá 3 giây thì backend tự xếp hạng chiến dịch (gợi ý) hoặc chấm điểm bằng luật nghiệp vụ (cảnh báo rủi ro).



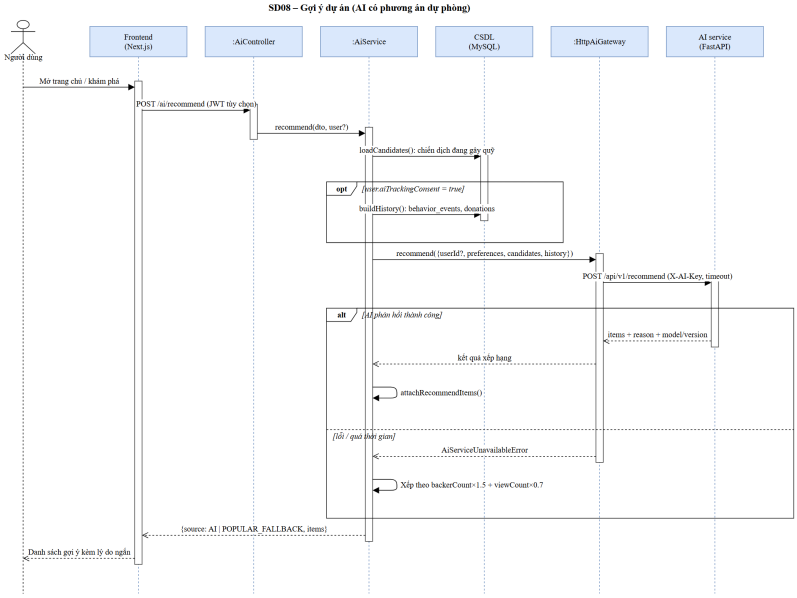
Hình 4.2 Sơ đồ tuần tự luồng tài trợ và xác nhận thanh toán (SD05)

Nhờ vậy tính năng nghiệp vụ không phụ thuộc vào việc AI sẵn sàng; giao diện ghi rõ kết quả đến từ mô hình hay từ luật dự phòng. Hành vi người dùng chỉ được gửi sang AI khi người dùng đã đồng ý cá nhân hóa. Trình tự được thể hiện trong Hình 4.3.

Các luồng còn lại (đăng ký, đăng nhập, xét duyệt chiến dịch, hoàn tiền, báo cáo vi phạm, tự động chốt chiến dịch) được mô tả trong bộ sơ đồ tuần tự SD01–SD11.

4.5 Thiết kế bảo mật

Mọi kiểm tra quyền và trạng thái tiền đều nằm ở backend. Các biện pháp bảo mật được tổ chức theo lớp như Bảng 4.4.



Hình 4.3 Sơ đồ tuần tự luồng gợi ý dự án có phương án dự phòng (SD08)

Bảng 4.4 Các biện pháp bảo mật theo lớp

Lớp	**Biện pháp**
Xác thực	Mật khẩu băm bcrypt; JWT access và refresh token; refresh token xoay vòng, dùng lại token cũ sẽ vô hiệu mọi phiên; thu hồi token theo `jti` khi đăng xuất; khóa tạm tài khoản sau 5 lần đăng nhập sai trong 15 phút
Xác minh bổ sung	Mã OTP qua email trước thao tác nhạy cảm: đổi mật khẩu, yêu cầu hoàn tiền, quản trị viên duyệt hoàn tiền và xuất danh sách tài khoản nhận hoàn
Phân quyền	Theo vai trò (`user`, `campaign_owner`, `admin`) và theo quyền sở hữu, kiểm tra ở máy chủ bằng `JwtAuthGuard`, `RolesGuard`; không thể tự đăng ký vai trò quản trị viên

Lớp	**Biện pháp**
Dữ liệu đầu vào	DTO và `ValidationPipe`; ORM tham số hóa truy vấn chống SQL injection; HTML mô tả được làm sạch ở máy chủ; kiểm tra loại và dung lượng ảnh tải lên
HTTP	`helmet`; CORS giới hạn nguồn qua `CORS_ORIGINS`; `trust proxy` theo số lớp proxy; giới hạn tần suất bằng Redis
Thanh toán	Webhook ký HMAC, chống phát lại bằng timestamp; IPN SePay xác thực khóa và đối chiếu số tiền; idempotency key; cập nhật có điều kiện chống xử lý trùng; không lưu thông tin thẻ
Nội bộ	Dịch vụ AI yêu cầu header `X-AI-Key`, so sánh hằng thời gian; trên AWS chỉ load balancer được gọi vào các task
Dữ liệu cá nhân	Không trả `passwordHash` ra API; ảnh giấy phép tổ chức không lộ ra API công khai; hành vi chỉ được ghi khi người dùng đồng ý, rút lại thì xóa; file thống kê xuất ra đã ẩn danh
Giao diện	Tham số `next` sau đăng nhập chỉ nhận đường dẫn nội bộ (`safeNext()`) để chống chuyển hướng mở; trạng thái thanh toán sau khi quay về từ SePay được tải lại từ máy chủ, không đọc từ URL; việc ẩn nút chỉ phục vụ trải nghiệm, quyền được kiểm tra lại ở backend
Truy vết	Nhật ký kiểm toán ghi thời điểm, người thực hiện, giá trị trước và sau; `X-Request-ID` đi xuyên suốt từ reverse proxy
Bí mật	Nằm trong biến môi trường; kho mã nguồn chỉ có `.env.example` với giá trị giữ chỗ

CHƯƠNG 5: TRIỂN KHAI, CHẤT LƯỢNG VÀ HẠN CHẾ

Hệ thống có hai môi trường chạy dùng chung một bộ Docker image: Docker Compose cho máy cá nhân và môi trường thử nghiệm, AWS được mô tả bằng Terraform cho môi trường đám mây.

5.1 Triển khai bằng Docker Compose

File `infra/compose.yaml` khởi động sáu dịch vụ (Bảng 5.1). Thứ tự khởi động dựa trên healthcheck: MySQL, Redis và dịch vụ AI sẵn sàng trước, sau đó tới backend, frontend và cuối cùng là Nginx.

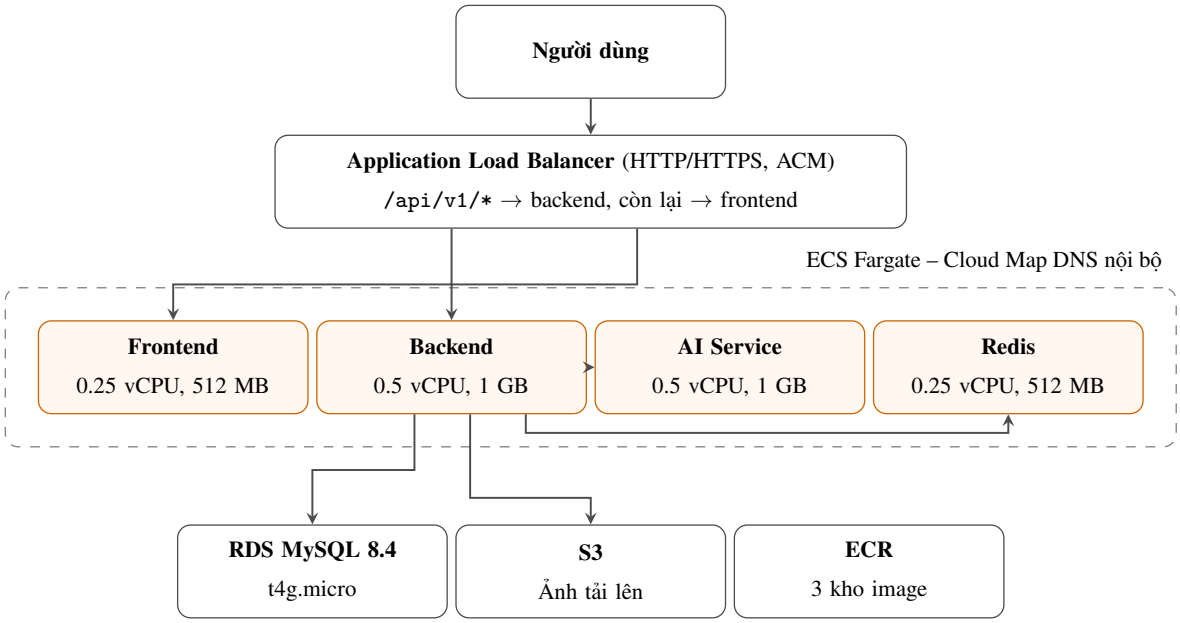
Bảng 5.1 Các dịch vụ trong Docker Compose

Dịch vụ	**Image / build**	**Cổng**	**Ghi chú**
`nginx`	nginx:1.27-alpine	8080 → 80	`/` → frontend, `/api/` → backend, `/ai/` → AI; giới hạn 10 MB, riêng khôi phục sao lưu 200 MB
`frontend`	`frontend/Dockerfile`	3000	Healthcheck `/api/health`
`backend`	`backend/Dockerfile`	4000	Healthcheck `/api/v1/health`; volume `uploads_data` lưu ảnh
`ai-service`	`ai-service/Dockerfile`	8000	Gắn thư mục `models/` chỉ đọc
`mysql`	mysql:8.4	3307 → 3306	Volume `mysql_data`
`redis`	redis:7-alpine	6379	Bật AOF, volume `redis_data`

5.2 Triển khai trên AWS

Thư mục `infra/terraform/` mô tả hạ tầng tại vùng `ap-southeast-1` (Singapore). Sơ đồ triển khai được trình bày trong Hình 5.1, chi tiết tài nguyên trong Bảng 5.2.

Quy trình triển khai gồm bốn bước: `terraform apply` tạo các kho ECR; build và đẩy ba image lên ECR (script `deploy.ps1`); `terraform apply` để tạo toàn bộ hạ tầng; `terraform destroy` khi không còn sử dụng để tránh phát sinh chi phí.



Hình 5.1 Sơ đồ triển khai trên AWS

Bảng 5.2 Tài nguyên AWS được tạo bằng Terraform

Tài nguyên	**Cấu hình**
VPC	2 public subnet trên 2 vùng sẵn sàng, Internet Gateway; không dùng NAT Gateway để tiết kiệm chi phí
Application Load Balancer	HTTP và HTTPS (chứng chỉ ACM); `/api/v1/*` chuyển tới backend, còn lại tới frontend
ECS Fargate	4 service, mỗi service 1 task: frontend 0.25 vCPU/512 MB, backend 0.5 vCPU/1 GB, AI 0.5 vCPU/1 GB, Redis 0.25 vCPU/512 MB
Cloud Map	DNS nội bộ cho `backend`, `ai`, `redis`; dịch vụ AI không mở ra load balancer
RDS	MySQL 8.4, lớp `t4g.micro`, `publicly_accessible = false`
S3	Bucket lưu ảnh, chặn truy cập công khai, mã hóa phía máy chủ; backend truy cập qua IAM task role
ECR	3 kho image: frontend, backend, ai-service
CloudWatch	Log group cho các task
Security Group	Chỉ load balancer được gọi vào các task; chỉ các task được gọi vào RDS

5.3 Thuộc tính chất lượng

Kiến trúc ưu tiên tính đúng của dữ liệu tiền và khả năng truy vết hơn hiệu năng tối đa, phù hợp với quy mô đồ án. Bảng 5.3 tóm tắt cách kiến trúc đáp ứng từng thuộc tính.

Bảng 5.3 Thuộc tính chất lượng và cách đáp ứng

Thuộc tính	**Cách kiến trúc đáp ứng**
Tính đúng giao dịch	Idempotency key, cập nhật có điều kiện, số liệu quỹ tính từ giao dịch đã xác minh, đối soát SePay khi lỡ IPN
Tính sẵn sàng	AI có dự phòng hai lớp; giới hạn tần suất tự lùi về bộ nhớ trong khi Redis lỗi; healthcheck cho Docker và load balancer
Khả năng mở rộng	Frontend, backend và AI không lưu trạng thái phiên nên có thể tăng số task; ảnh lưu trên S3 không phụ thuộc ổ đĩa cục bộ
Khả năng bảo trì	Module nghiệp vụ độc lập; tích hợp qua interface; migration có rollback; OpenAPI làm hợp đồng giữa các thành phần
Khả năng truy vết	Nhật ký kiểm toán trước/sau; `X-Request-ID`; log tập trung trên CloudWatch
Hiệu năng	Phân trang và lọc phía máy chủ; SSR cho trang công khai

5.4 Hạn chế và hướng mở rộng

- Mỗi service trên AWS chỉ có một task và chưa cấu hình tự động mở rộng. Hướng mở rộng: bật ECS Service Auto Scaling theo CPU hoặc số yêu cầu.
- Redis chạy dưới dạng task Fargate thay vì ElastiCache; khởi động lại thì mất bộ đếm giới hạn tần suất.
- Tác vụ định kỳ chạy trong mọi instance backend. Khi tăng số task cần thêm khóa phân tán hoặc tách thành một worker riêng.
- File mô hình `.pkl` không nằm trong Git nên môi trường mới chạy bằng luật dự phòng cho tới khi huấn luyện lại.
- Chưa có hàng đợi thông điệp; email được gửi trực tiếp từ process backend.
- Hoàn tiền các khoản qua SePay phải chuyển khoản thủ công vì SePay không cung cấp API hoàn tiền.

5.5 Kết luận

Kiến trúc của Góp Mầm tách rõ ba thành phần ứng dụng, đặt toàn bộ nghiệp vụ và trạng thái tiền tại backend, đóng gói mọi tích hợp bên ngoài qua interface và giữ cho AI ở vai trò hỗ trợ có phương án dự phòng. Cùng một bộ image chạy được trên Docker Compose và AWS giúp nhóm phát triển, kiểm thử và triển khai trên cùng một cấu hình. Các hạn chế nêu trên chủ yếu liên quan tới vận hành ở quy mô lớn và không ảnh hưởng tới các chức năng trong phạm vi đồ án.

TÀI LIỆU THAM KHẢO

Amazon Web Services. (2026). *Amazon elastic container service developer guide.* https://docs.aws.amazon.com/ecs/.

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* (Unpublished doctoral dissertation). University of California, Irvine.

HashiCorp. (2026). *Terraform aws provider documentation.* https://registry.terraform.io/providers/hashicorp/aws/latest/docs.

Jones, M. B., Bradley, J., & Sakimura, N. (2015, May). *Json web token (jwt)* (RFC No. 7519). Internet Engineering Task Force. (Available at: https://www.rfc-editor.org/info/rfc7519)

Krawczyk, H., Bellare, M., & Canetti, R. (1997, February). *Hmac: Keyed-hashing for message authentication* (RFC No. 2104). Internet Engineering Task Force. (Available at: https://www.rfc-editor.org/info/rfc2104)

NestJS. (2026). *Nestjs documentation.* https://docs.nestjs.com.

Oracle Corporation. (2026). *Mysql 8.4 reference manual.* https://dev.mysql.com/doc/refman/8.4/en/.

OWASP Foundation. (2021). *Owasp top 10 web application security risks.* https://owasp.org/www-project-top-ten/.

Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, *12*, 2825–2830.

Ramírez, S. (2026). *Fastapi documentation.* https://fastapi.tiangolo.com.

Redis Ltd. (2026). *Redis documentation.* https://redis.io/docs/.

SePay. (2026). *Tài liệu tích hợp cổng thanh toán sepay.* https://docs.sepay.vn.

Vercel. (2026). *Next.js documentation.* https://nextjs.org/docs.